

Exercise 01:

Write a program that reads a Celsius degree and converts it to Fahrenheit.

Formula: $F = (9/5) * Celsius + 32$

```
#include <iostream>
using namespace std;
int main() {
    double celsius;
    cout << "Enter a degree in Celsius: ";
    cin >> celsius;
    double fahrenheit = (9.0 / 5) * celsius + 32;
    cout << celsius << " in Celsius is "
        << fahrenheit << " Fahrenheit";
    return 0;
}
```

Output:

```
Enter a degree in Celsius: 43
43 in Celsius is 109.4 Fahrenheit
```

Exercise 02:

Write a program that asks the user to enter the starting velocity v0, the ending velocity v1 and the time span t, and then displays the average acceleration with two digits after the decimal point using the formula:

$a = (v1 - v0) / t$

```
#include <iostream>
#include <iomanip>
using namespace std;
int main() {
    double v0, v1, t;
    cout << "Enter v0: ";
    cin >> v0;
    cout << "Enter v1: ";
    cin >> v1;
    cout << "Enter t: ";
    cin >> t;
    double a = (v1 - v0) / t;
    cout << fixed << setprecision(2);
    cout << "The average acceleration is " << a;
    return 0;
}
```

Output:

```
Enter v0: 5.5
Enter v1: 50.9
Enter t: 4.5
The average acceleration is 10.09
```

Exercise 03:

Write a program that displays the following on the screen:

```
1 1
2 4
3 9
4 16
5 25
```

You should use pow function.

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    for (int i = 1; i <= 5; i++) {
        cout << i << " " << (int)pow(i, 2) << endl;
    }
    return 0;
}
```

Exercise 04:

Write a program that asks the user to enter a radius of a circle and then displays its area and its perimeter.

Area = $\text{PI} \times r^2$

Perimeter = $2 \times \text{PI} \times r$

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    double radius;
    cout << "Enter the radius: ";
    cin >> radius;
    double area = M_PI * pow(radius, 2);
    double perimeter = 2 * M_PI * radius;
    cout << "Area = " << area << endl;
    cout << "Perimeter = " << perimeter;
    return 0;
}
```

Output:

```
Enter the radius: 5
Area = 78.5398
Perimeter = 31.4159
```

Exercise 05:

Write a program that asks the user to enter character and then displays its ASCII code.

```
#include <iostream>
using namespace std;
int main() {
    char ch;
    cout << "Enter a character: ";
    cin >> ch;
    cout << "ASCII code = " << (int)ch;
    return 0;
}
```

Output:

```
Enter a character: A
ASCII code = 65
```

Exercise 06:

Given the following:

One birth every 7 seconds

One death every 13 seconds

One new immigrant every 45 seconds

Current population: 1000000

One year: 365 days

Write a program that displays the population for each of the next five years.

```
#include <iostream>
using namespace std;
int main() {
    int population = 1000000;
    int birthsPerYear = (365 * 24 * 60 * 60) / 7;
    int deathsPerYear = (365 * 24 * 60 * 60) / 13;
    int immigrantsPerYear = (365 * 24 * 60 * 60) / 45;
    for (int year = 1; year <= 5; year++) {
        population = population + birthsPerYear - deathsPerYear + immigrantsPerYear;
        cout << "Year " << year << ": " << population << endl;
    }
    return 0;
}
```

Output:

```
Year 1: 2861285
Year 2: 4722570
Year 3: 6583855
Year 4: 8445140
Year 5: 10306425
```

Exercise 07:

Write a program that asks the user to enter an integer and then displays if it is even or odd.

You should use switch case.

```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "Enter an integer: ";
    cin >> n;
    switch (n % 2) {
        case 0:
            cout << "Even";
            break;
        case 1:
        case -1:
            cout << "Odd";
            break;
    }
    return 0;
}
```

Output:

```
Enter an integer: 12
Even
```

Exercise 08:

Write a program that reads an arithmetic operation op (+, -, * or /) and two numbers x and y, and performs the operation x op y. Two solutions: if and switch.

Solution 1: Using Switch

```
#include <iostream>
using namespace std;
int main() {
    char op;
    double x, y;
    cout << "Enter an operation: ";
    cin >> op;
    cout << "Enter x: ";
    cin >> x;
    cout << "Enter y: ";
    cin >> y;
    switch(op) {
        case '+':
            cout << x << " + " << y << " = " << x + y;
            break;
        case '-':
            cout << x << " - " << y << " = " << x - y;
            break;
        case '*':
            cout << x << " * " << y << " = " << x * y;
            break;
        case '/':
            cout << x << " / " << y << " = " << x / y;
            break;
        default:
            cout << "Invalid operation";
    }
    return 0;
}
```

Output:

```
Enter an operation: *
Enter x: 2
Enter y: 5
2 * 5 = 10
```

Solution 2: Using If

```
#include <iostream>
using namespace std;
int main() {
    char op;
    double x, y;
    cout << "Enter an operation: ";
    cin >> op;
    cout << "Enter x: ";
    cin >> x;
    cout << "Enter y: ";
    cin >> y;
```

```

if (op == '+')
    cout << x << " + " << y << " = " << x + y;
else if (op == '-')
    cout << x << " - " << y << " = " << x - y;
else if (op == '*')
    cout << x << " * " << y << " = " << x * y;
else if (op == '/')
    cout << x << " / " << y << " = " << x / y;
else
    cout << "Invalid operation";
return 0;
}

```

Output:

```

Enter an operation: -
Enter x: 20
Enter y: 45.5
20 - 45.5 = -25.5

```

Exercise 09:

Write a method that computes the sum of the digits in an integer.

int sumDigits(long n)

Example , sumDigits(234) returns 9

```

#include <iostream>
using namespace std;
int sumDigits(long n) {
    int sum = 0;
    while (n != 0) {
        sum += n % 10;
        n /= 10;
    }
    return sum;
}
int main() {
    long number;
    cout << "Enter an integer: ";
    cin >> number;
    cout << "Sum of digits = " << sumDigits(number);
    return 0;
}

```

Output:

```

Enter an integer: 234
Sum of digits = 9

```

Exercise 10:

A palindromic prime is a prime number and palindromic. Write a program that displays the first 100 palindromic prime numbers. Display 10 numbers per line, separated by exactly one space.

```
#include <iostream>
#include <cmath>
using namespace std;
bool isPrime(int n) {
    if (n < 2)
        return false;
    for (int i = 2; i <= sqrt(n); i++) {
        if (n % i == 0)
            return false;
    }
    return true;
}
bool isPalindrome(int n) {
    int original = n;
    int reverse = 0;
    while (n != 0) {
        reverse = reverse * 10 + n % 10;
        n /= 10;
    }
    return original == reverse;
}
int main() {
    int count = 0;
    int number = 2;
    while (count < 100) {
        if (isPrime(number) && isPalindrome(number)) {
            cout << number << " ";
            count++;
            if (count % 10 == 0)
                cout << endl;
        }
        number++;
    }
    return 0;
}
```

Output (First Lines):

```
2 3 5 7 11 101 131 151 181 191
313 353 373 383 727 757 787 797 919 929
```

Exercise 11:

- Create an array of integers named List. Use the shorthand notation to fill it with ten values.
- Write the code to sum up all the elements in the array.
- Write the code to find the highest value in the array.

```
#include <iostream>
using namespace std;
int main() {
    int list[] = {12, 5, 18, 9, 25, 7, 30, 14, 3, 20};
    int sum = 0;
    for (int i = 0; i < 10; i++) {
        sum += list[i];
    }
    cout << "Sum = " << sum << endl;
    int max = list[0];
    for (int i = 1; i < 10; i++) {
        if (list[i] > max)
            max = list[i];
    }
    cout << "Highest value = " << max;
    return 0;
}
```

Output:

```
Sum = 143
Highest value = 30
```

Exercise 12:

Write methods to:

Find the average of array elements.

Count elements above/below a key.

Find the range between maximum and minimum.

Test the methods by entering 10 numbers.

```
#include <iostream>
using namespace std;
double average(double array[], int size) {
    double sum = 0;
    for (int i = 0; i < size; i++)
        sum += array[i];
    return sum / size;
}
int countAbove(double array[], int size, double key) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (array[i] > key)
            count++;
    }
    return count;
}
int countBelow(double array[], int size, double key) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (array[i] < key)
            count++;
    }
    return count;
}
```

```

}
double findRange(double array[], int size) {
    double max = array[0];
    double min = array[0];
    for (int i = 1; i < size; i++) {
        if (array[i] > max)
            max = array[i];
        if (array[i] < min)
            min = array[i];
    }
    return max - min;
}
int main() {
    double numbers[10];
    cout << "Enter 10 numbers:" << endl;
    for (int i = 0; i < 10; i++)
        cin >> numbers[i];
    double avg = average(numbers, 10);
    cout << "Average = " << avg << endl;
    cout << "Above Average = " << countAbove(numbers,10,avg) << endl;
    cout << "Below Average = " << countBelow(numbers,10,avg) << endl;
    cout << "Range = " << findRange(numbers,10);
    return 0;
}

```

Output:

```

Enter 10 numbers:
5
7
3
10
8
4
2
6
9
1
Average = 5.5
Above Average = 5
Below Average = 5
Range = 9

```

Exercise 13:

UML Diagram:

Triangle
- side1 : double - side2 : double - side3 : double
+ Triangle() + Triangle(double,double,double) + area() : double + perimeter() : double


```

#include <cmath>
class Triangle {
private:
    double side1;
    double side2;
    double side3;
public:
    Triangle() {
        side1 = side2 = side3 = 1.0;
    }
    Triangle(double s1, double s2, double s3) {
        side1 = s1;
        side2 = s2;
        side3 = s3;
    }
    double perimeter() {
        return side1 + side2 + side3;
    }
    double area() {
        double s = perimeter() / 2;
        return sqrt(s * (s-side1) * (s-side2) * (s-side3));
    }
};

```

Test Program:

```

#include <iostream>
using namespace std;
int main() {
    Triangle t1;
    Triangle t2(4,5,6);
    cout << "Triangle 1" << endl;
    cout << "Area = " << t1.area() << endl;
    cout << "Perimeter = " << t1.perimeter() << endl;
    cout << endl;
    cout << "Triangle 2" << endl;
    cout << "Area = " << t2.area() << endl;
    cout << "Perimeter = " << t2.perimeter();
    return 0;
}

```

Output:

```

Triangle 1
Area = 0.433013
Perimeter = 3
Triangle 2
Area = 9.92157
Perimeter = 15

```

Exercise 14:

[The Account class]

Design a class named Account that contains:

Private data fields:

Id balance annualInterestRate dateCreated

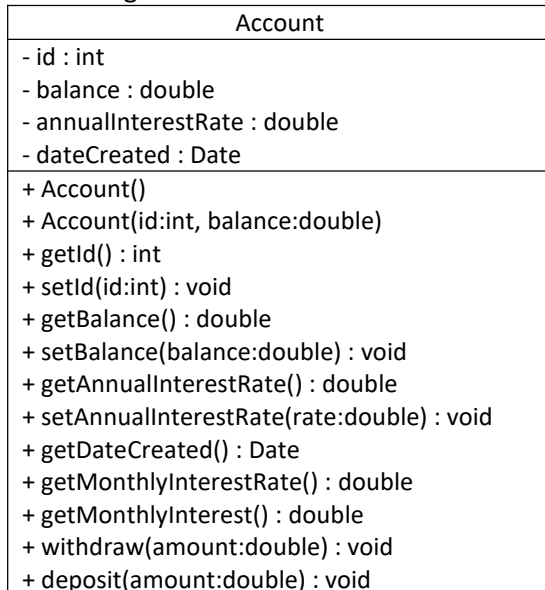
Constructors:

Default constructor Constructor with id and balance

Methods:

Getters and setters Monthly interest rate Monthly interest Withdraw Deposit

UML Diagram:



```
#include <iostream>
#include <ctime>
using namespace std;
class Account {
private:
    int id;
    double balance;
    double annualInterestRate;
    time_t dateCreated;
public:
    Account() {
        id = 0;
        balance = 0;
        annualInterestRate = 0;
        dateCreated = time(0);
    }
    Account(int id, double balance) {
        this->id = id;
        this->balance = balance;
        annualInterestRate = 0;
        dateCreated = time(0);
    }
    int getId() {
        return id;
    }
}
```

```

void setId(int id) {
    this->id = id;
}
double getBalance() {
    return balance;
}
void setBalance(double balance) {
    this->balance = balance;
}
double getAnnualInterestRate() {
    return annualInterestRate;
}
void setAnnualInterestRate(double rate) {
    annualInterestRate = rate;
}
time_t getDateCreated() {
    return dateCreated;
}
double getMonthlyInterestRate() {
    return annualInterestRate / 100 / 12;
}
double getMonthlyInterest() {
    return balance * getMonthlyInterestRate();
}
void withdraw(double amount) {
    balance -= amount;
}
void deposit(double amount) {
    balance += amount;
}
};

```

Test Program:

```

#include <iostream>
#include <ctime>
using namespace std;
int main() {
    Account account(1122,20000);
    account.setAnnualInterestRate(4.5);
    account.withdraw(2500);
    account.deposit(3000);
    cout << "Balance: $" << account.getBalance() << endl;
    cout << "Monthly Interest: $" << account.getMonthlyInterest() << endl;
    cout << "Date Created: " << ctime(&account.getDateCreated());
    return 0;
}

```

Output:

```

Balance: $20500
Monthly Interest: $76.875
Date Created: Fri Jul 24 09:00:00 2026

```

Exercise 15:

[Substring]

Write your own method to check whether the first string is a substring of the second string.

```
#include <iostream>
#include <string>
using namespace std;
bool isSubstring(string s1, string s2) {
    return s2.find(s1) != string::npos;
}
int main() {
    string str1;
    string str2;
    cout << "Enter first string: ";
    getline(cin, str1);
    cout << "Enter second string: ";
    getline(cin, str2);
    if(isSubstring(str1, str2))
        cout << "The first string is a substring of the second";
    else
        cout << "The first string is not a substring of the second";
    return 0;
}
```

Output 1:

```
Enter first string: Java
Enter second string: I love Java programming
The first string is a substring of the second
```

Output 2:

```
Enter first string: Python
Enter second string: I love Java programming
The first string is not a substring of the second
```

Exercise 16:

[Inheritance]

Create a class Person and a subclass Student that inherits from Person.

```
#include <iostream>
#include <string>
using namespace std;
class Person {
private:
    string name;
    int age;
public:
    Person(string name, int age) {
        this->name = name;
        this->age = age;
    }
    virtual void displayInfo() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};
Student Class (Inheritance):
class Student : public Person {
```

```

private:
    string major;
public:
    Student(string name, int age, string major) : Person(name, age) {
        this->major = major;
    }
    void displayInfo() override {
        Person::displayInfo();
        cout << "Major: " << major << endl;
    }
};

```

Test Program:

```

int main() {
    Person p1("Ali",25);
    Student s1("Ahmad",21,"Software Engineering");
    cout << "Person Information:" << endl;
    p1.displayInfo();
    cout << endl;
    cout << "Student Information:" << endl;
    s1.displayInfo();
    return 0;
}

```

Output:

```

Person Information:
Name: Ali
Age: 25

Student Information:
Name: Ahmad
Age: 21
Major: Software Engineering

```

Exercise 17:

```

#include <iostream>
#include <vector>
using namespace std;
class GeometricObject {
public:
    virtual void display() {
        cout << "Geometric Object" << endl;
    }
    virtual ~GeometricObject(){}
};
class Shape : public GeometricObject {
private:
    int size;
    string color;
    bool filled;
public:
    Shape(int s, string c, bool f) {
        size = s;
        color = c;
        filled = f;
    }
}

```

```

void display() override {
    cout << "Shape size: " << size
        << ", Color: " << color
        << ", Filled: " << filled << endl;
}
};
vector<Shape*> getShapes(GeometricObject* g[], int size) {
    vector<Shape*> shapes;
    for(int i=0;i<size;i++) {
        Shape* s = dynamic_cast<Shape*>(g[i]);
        if(s != nullptr)
            shapes.push_back(s);
    }
    return shapes;
}
int main() {
    GeometricObject* g[10];
    for(int i=0;i<10;i+=2) {
        g[i] = new GeometricObject();
        g[i+1] = new Shape(3+i,"Red",false);
    }
    vector<Shape*> s = getShapes(g,10);
    for(int i=0;i<s.size();i++)
        s[i]->display();
    return 0;
}

```

Exercise 18: ArrayList

```

#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> numbers;
    // Add elements
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);
    numbers.push_back(40);
    numbers.push_back(50);
    cout << "ArrayList: ";
    for(int n : numbers)
        cout << n << " ";
    cout << endl;
    // Remove element index 2
    numbers.erase(numbers.begin()+2);
    cout << "After removing index 2: ";
    for(int n : numbers)
        cout << n << " ";
    cout << endl;
    int n;
    cout << "Enter a number to search: ";
    cin >> n;
    bool found=false;
    for(int x : numbers) {
        if(x==n)
            found=true;
    }
}

```

```

}
if(found)
    cout << "Number exists";
else
    cout << "Number does not exist";
return 0;
}

```

Output:

```

ArrayList: 10 20 30 40 50
After removing index 2: 10 20 40 50
Enter a number to search: 40
Number exists

```

Exercise 19: Exception Handling

```

#include <iostream>
using namespace std;
int main() {
    try {
        int n1,n2;
        cout << "Enter first number: ";
        cin >> n1;
        cout << "Enter second number: ";
        cin >> n2;
        if(n2==0)
            throw runtime_error("Cannot divide by zero");
        int result = n1 / n2;
        cout << "Result = " << result;
    }
    catch(runtime_error e) {
        cout << "Error: " << e.what();
    }
    catch(...) {
        cout << "Invalid input";
    }
    cout << endl;
    cout << "Program finished";
    return 0;
}

```

Output 1:

```

Enter first number: 20
Enter second number: 5
Result = 4
Program finished

```

Output 2:

```

Enter first number: 20
Enter second number: 0
Error: Cannot divide by zero
Program finished

```

Exercise 20: Text I/O and File Handling

```
#include <iostream>
#include <fstream>
using namespace std;
int main() {
    // Writing to file
    ofstream writer("students.txt");
    if(writer.is_open()) {
        writer << "Ali\n";
        writer << "Ahmad\n";
        writer << "Sara\n";
        writer.close();
        cout << "File created successfully" << endl;
    }
    else {
        cout << "Error writing file" << endl;
    }
    // Reading from file
    ifstream reader("students.txt");
    if(reader.is_open()) {
        string name;
        cout << "Students:" << endl;
        while(getline(reader,name)) {
            cout << name << endl;
        }
        reader.close();
    }
    else {
        cout << "File not found";
    }
    return 0;
}
```

Output:

```
File created successfully
Students:
Ali
Ahmad
Sara
```

Exercise 21: Enum in C++

Create an enum named Day that contains the days of the week.

Write a program that asks the user to enter a day and displays whether it is a weekday or weekend using enum.

```
#include <iostream>
#include <string>
using namespace std;
enum Day {
    MONDAY,
    TUESDAY,
    WEDNESDAY,
    THURSDAY,
    FRIDAY,
    SATURDAY,
    SUNDAY
};
```



```

int main() {
    string dayInput;
    cout << "Enter a day: ";
    cin >> dayInput;
    for(char &c : dayInput)
        c = toupper(c);
    Day day;
    if(dayInput == "MONDAY")
        day = MONDAY;
    else if(dayInput == "TUESDAY")
        day = TUESDAY;
    else if(dayInput == "WEDNESDAY")
        day = WEDNESDAY;
    else if(dayInput == "THURSDAY")
        day = THURSDAY;
    else if(dayInput == "FRIDAY")
        day = FRIDAY;
    else if(dayInput == "SATURDAY")
        day = SATURDAY;
    else
        day = SUNDAY;
    switch(day) {
        case SATURDAY:
        case SUNDAY:
            cout << "Weekend";
            break;
        default:
            cout << "Weekday";
    }
    return 0;
}

```

Output 1:

```

Enter a day: Monday
Weekday

```

Output 2:

```

Enter a day: Sunday
Weekend

```

Exercise 22:

```

#include <iostream>
#include <list>
using namespace std;
bool isAnagram(list<char> w1, list<char> w2) {
    if(w1.size() != w2.size())
        return false;
    list<char> temp = w2;
    for(char c : w1) {
        auto it = find(temp.begin(), temp.end(), c);
        if(it != temp.end())
            temp.erase(it);
        else
            return false;
    }
    return true;
}

```

```

}
int main() {
    list<char> word1 = {'t','r','e','a','s','o','n'};
    list<char> word2 = {'s','e','n','a','t','o','r'};
    cout << boolalpha << isAnagram(word1,word2);
    return 0;
}

```

Output:

true

Exercise 23:

```

#include <iostream>
#include <list>
using namespace std;
class MyLinkedList {
public:
    list<int> numbers;
    int sumValues() {
        int sum = 0;
        for(int value : numbers)
            sum += value;
        return sum;
    }
};
int main() {
    MyLinkedList obj;
    obj.numbers.push_back(6);
    obj.numbers.push_back(4);
    obj.numbers.push_back(10);
    obj.numbers.push_back(9);
    obj.numbers.push_back(1);
    cout << "Sum = " << obj.sumValues();
    return 0;
}

```

Output:

Sum = 30

Exercise 24:

sumNumbers() method receives a linked list as parameter and returns the sum.

```

#include <iostream>
#include <list>
using namespace std;
int sumNumbers(list<int> numbers) {
    int sum = 0;
    for(int number : numbers)
        sum += number;
    return sum;
}
int main() {
    list<int> numbers;
    numbers.push_back(6);
    numbers.push_back(4);
    numbers.push_back(10);
    numbers.push_back(9);
}

```

```

numbers.push_back(1);
cout << "Sum = " << sumNumbers(numbers);
return 0;
}

```

Output:

```
Sum = 30
```

Exercise 25:

```

#include <iostream>
#include <stack>
#include <string>
using namespace std;
bool check(string expression) {
    stack<char> stack;
    for(char ch : expression) {
        if(ch == '(') {
            stack.push(ch);
        }
        else if(ch == ')') {
            if(stack.empty())
                return false;
            stack.pop();
        }
    }
    return stack.empty();
}
int main() {
    string exp;
    cout << "Enter expression: ";
    getline(cin, exp);
    if(check(exp))
        cout << "Balanced expression";
    else
        cout << "Not balanced expression";
    return 0;
}

```

Output 1:

```
Enter expression: (((a+b)*c)+(u*v))
Balanced expression
```

Output 2:

```
Enter expression: (a+b)*c)
Not balanced expression
```

Exercise 26:

In a bank, customers wait in a queue (q0) for their turn to complete a transaction. Customers who come first are served first.

At a certain point when there are lots of customers waiting, the queue is split into two empty queues q1 and q2 based on the alphabetical order of their names.

Customers with first name starting A-H are added to q1 while customers with first name starting I-Z are added to q2.

Write a method named splitQueue that accepts 3 queues:

q0: original queue

q1: empty queue

q2: empty queue

The method should split q0 elements into q1 and q2.

```
#include <iostream>
#include <queue>
#include <string>
using namespace std;
void splitQueue(queue<string>& q0, queue<string>& q1, queue<string>& q2) {
    while(!q0.empty()) {
        string name = q0.front();
        q0.pop();
        char firstLetter = toupper(name[0]);
        if(firstLetter >= 'A' && firstLetter <= 'H') {
            q1.push(name);
        }
        else {
            q2.push(name);
        }
    }
}
int main() {
    queue<string> q0;
    queue<string> q1;
    queue<string> q2;
    int n;
    cout << "Enter number of customers: ";
    cin >> n;
    cin.ignore();
    for(int i = 0; i < n; i++) {
        string name;
        cout << "Enter customer name: ";
        getline(cin, name);
        q0.push(name);
    }
    splitQueue(q0, q1, q2);
    cout << "\nQueue 1 (A-H):" << endl;
    while(!q1.empty()) {
        cout << q1.front() << endl;
        q1.pop();
    }
    cout << "\nQueue 2 (I-Z):" << endl;
    while(!q2.empty()) {
        cout << q2.front() << endl;
        q2.pop();
    }
    return 0;
}
```

Output:

Enter number of customers: 6

Enter customer name: Ali

Enter customer name: Sara

Enter customer name: Hassan

Enter customer name: Omar

Enter customer name: Ahmad

Enter customer name: Lina

Queue 1 (A-H):

Ali

Hassan

Ahmad

Queue 2 (I-Z):

Sara

Omar

Lina

Exercise 27:

[Big-O Proof]

Prove that $n^3 + 1000 = O(n^3)$

According to Big-O definition $f(n) \leq c * g(n)$

Where $f(n) = n^3 + 1000$ $g(n) = n^3$

Choose:

$c = 2$

$n_0 = 10$

For every:

$n \geq 10$

we have:

$n^3 \geq 1000$

Therefore:

$n^3 + 1000 \leq n^3 + n^3$ so $n^3 + 1000 \leq 2n^3$

This means: $f(n) \leq c * g(n)$ therefore $n^3 + 1000 = O(n^3)$ final Answer: $c = 2$ $n_0 = 10$

Exercise 28:

Determine Running Time using Big-O

```
void method(int n) {  
    int a[n];  
    for(int i = 0; i < n; i++) {  
        a[i] = input;  
    }  
    for(int i = 0; i < n; i++) {  
        cout << a[i];  
    }  
}
```

Step 1: Creating the array:

int a[n] (Time complexity $O(1)$)

Step 2: First loop:

for(int i = 0; i < n; i++) (The loop runs n times $O(n)$)

Step 3: Second loop:

for(int i = 0; i < n; i++) (The loop runs n times $O(n)$)

Step 4: Total running time

$(O(1) + O(n) + O(n) \rightarrow O(n))$

Exercise 29:

[Recursive oddSum Method]

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example:

oddSum(5) = 1 + 3 + 5 = 9

```
#include <iostream>
using namespace std;
int oddSum(int n) {
    if(n == 1)
        return 1;
    return n + oddSum(n - 2);
}
int main() {
    int n;
    cout << "Enter an odd number: ";
    cin >> n;
    cout << "Sum = " << oddSum(n);
    return 0;
}
```

Output:

```
Enter an odd number: 7
Sum = 16
```

Exercise 30:

[Recursive oddSum with Debugging]

Write a recursive method named oddSum that takes a positive odd integer n and returns the sum of odd integers from 1 to n.

Example:

oddSum(7)=1+3+5+7=16

```
#include <iostream>
using namespace std;
int oddSum(int n) {
    cout << "oddSum called with n = "
        << n << endl;
    if(n == 1)
        return 1;
    return n + oddSum(n - 2);
}
int main()
{
    int n;
    cout << "Enter a positive odd number: ";
    cin >> n;
    cout << "Sum of odd numbers = "
        << oddSum(n);
    return 0;
}
```

Output:

Enter a positive odd number: 9

oddSum called with n = 9
oddSum called with n = 7
oddSum called with n = 5
oddSum called with n = 3
oddSum called with n = 1

Sum of odd numbers = 25

Exercise 31:**[Recursive Palindrome]**

A palindrome is a word that reads the same forward and backward.

Example: otto , level , radar

Write a recursive method isPalindrome that returns true if the word is palindrome.

```
#include <iostream>
#include <string>
using namespace std;
bool isPalindrome(string word) {
    if(word.length() <= 1)
        return true;
    if(word[0] != word[word.length()-1])
        return false;
    return isPalindrome(word.substr(1, word.length()-2));
}
int main() {
    string word;
    cout << "Enter a word: ";
    getline(cin, word);
    if(isPalindrome(word))
        cout << "The word is a palindrome";
    else
        cout << "The word is not a palindrome";
    return 0;
}
```

Output 1:

Enter a word: otto
The word is a palindrome

Output 2:

Enter a word: hello
The word is not a palindrome

Exercise 32:

[Full Binary Tree using Recursion]

A full binary tree is a tree where every node has either:

Two children

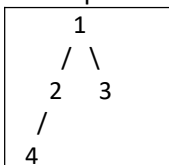
No children (leaf)

```
#include <iostream>
using namespace std;
class TreeNode {
public:
    int value;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int value) {
        this->value = value;
        left = nullptr;
        right = nullptr;
    }
};
bool isFullBinaryTree(TreeNode* root) {
    if(root == nullptr)
        return true;
    if(root->left == nullptr && root->right == nullptr)
        return true;
    if(root->left == nullptr || root->right == nullptr)
        return false;
    return isFullBinaryTree(root->left) && isFullBinaryTree(root->right);
}
int main() {
    TreeNode* root = new TreeNode(1);
    root->left = new TreeNode(2);
    root->right = new TreeNode(3);
    root->left->left = new TreeNode(4);
    root->left->right = new TreeNode(5);
    root->right->left = new TreeNode(6);
    root->right->right = new TreeNode(7);
    if(isFullBinaryTree(root))
        cout << "The tree is a full binary tree";
    else
        cout << "The tree is not a full binary tree";
    return 0;
}
```

Output:

The tree is a full binary tree

Example NOT Full Binary Tree:



Output:

The tree is not a full binary tree

Exercise 33:

[Graph Traversal (DFS)]

Create a graph using adjacency lists and implement Depth First Search.

```
#include <iostream>
#include <vector>
using namespace std;
class Graph {
private:
    int vertices;
    vector<int>* adjacencyList;
public:
    Graph(int vertices) {
        this->vertices = vertices;
        adjacencyList = new vector<int>[vertices];
    }
    void addEdge(int source, int destination) {
        adjacencyList[source].push_back(destination);
    }
    void DFS(int start) {
        bool* visited = new bool[vertices]{false};
        DFSUtil(start, visited);
    }
    void DFSUtil(int vertex, bool visited[]) {
        visited[vertex] = true;
        cout << vertex << " ";
        for(int neighbor : adjacencyList[vertex]) {
            if(!visited[neighbor])
                DFSUtil(neighbor, visited);
        }
    }
};
int main() {
    Graph graph(5);
    graph.addEdge(0,1);
    graph.addEdge(0,2);
    graph.addEdge(1,3);
    graph.addEdge(2,4);
    cout << "DFS Traversal:" << endl;
    graph.DFS(0);
    return 0;
}
```

Output:

```
DFS Traversal:
0 1 3 2 4
```

Exercise 34:

[HashMap]

```
#include <iostream>
#include <unordered_map>
#include <string>
using namespace std;
int main() {
    unordered_map<string,int> students;
    students["Ali"] = 90;
    students["Ahmad"] = 85;
```

```

students["Sara"] = 95;
cout << "Students Grades:" << endl;
for(auto student : students) {
    cout << student.first
        << "="
        << student.second
        << endl;
}
string name;
cout << "Enter student name: ";
cin >> name;
if(students.find(name) != students.end())
    cout << "Grade = " << students[name];
else
    cout << "Student not found";
return 0;
}

```

Output:

```

Students Grades:
Ali=90
Ahmad=85
Sara=95

```

```

Enter student name:
Sara

```

```

Grade = 95

```

Exercise 35:

[Vector]

```

#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> numbers;
    numbers.push_back(10);
    numbers.push_back(20);
    numbers.push_back(30);
    numbers.push_back(40);
    cout << "Vector: ";
    for(int n : numbers)
        cout << n << " ";
    cout << endl;
    numbers.erase(numbers.begin()+2);
    cout << "After removing element: ";
    for(int n : numbers)
        cout << n << " ";
    cout << endl;
    cout << "First element: "
        << numbers[0];
    return 0;
}

```

Output:

Vector: 10 20 30 40

After removing element: 10 20 40